



Google Developer Groups

On Campus • Fr. Conceicao Rodrigues College of Engineering

AROUND THE WORLD

BIT N BUILD

INTERNAL ROUND





Google Developer Groups

On Campus • Fr. Conceicao Rodrigues College of Engineering

AROUND THE WORLD



Web / App Dev

1. CreatorAi: AI-Powered Creator Operating Platform

Backstory:

The creator economy is growing rapidly, but content creation remains fragmented across tools for scripting, asset storage, editing, scheduling, and publishing. This leads to scattered assets, repeated work, context switching, and inefficient content production.

Problem Statement:

Develop CreatorAi, an AI-powered content operations platform that unifies the creator workflow from idea and script to recording, editing, repurposing, and publishing. The system should understand both textual and visual content, intelligently organize creator assets, and automate repetitive production tasks. The platform must be capable of taking a script and corresponding raw footage, identifying relevant sections, generating short-form clips, creating hooks and supporting content, and adapting the resulting content for different platforms. AI-generated edits should remain editable so that creators retain control over the final output.

Key Features:

- **Asset Management:** Centralized management of videos, images, audio, and other creator assets.
- **AI Script & Hook Generation:** Generate hooks, scripts, and supporting content.
- **Script-to-Video Understanding:** Understand the relationship between scripts and corresponding footage.
- **Automated Clip Generation:** Identify relevant sections of long-form footage and generate short-form clips.
- **AI-Assisted Editing:** Apply AI-generated edits while keeping the resulting content editable.
- **Multi-Platform Adaptation:** Adapt content for different platforms and formats.
- **Content Workflow:** Manage the content lifecycle from idea to publishing.
- **Creator Intelligence:** Provide insights into content performance and production patterns.

2. Roundtable: Live Captions for Group Conversations

Backstory:

Live captioning works well when there is one clear speaker and a close microphone. Real conversations are much harder. In meetings, classrooms, and cafés, multiple people may speak from different positions, overlap with each other, and be surrounded by background noise. Each device therefore captures a different and imperfect view of the same conversation, making accurate transcription and speaker identification difficult.

Problem Statement:

Develop Roundtable, a web application that uses multiple nearby devices to provide low-latency, speaker-attributed captions for a shared conversation. Participants should be able to join a common session without special hardware. The system must combine information from multiple devices to improve understanding of the conversation, handle overlapping speakers and background noise, and associate spoken segments with the correct participant.

Key Features:

- **Multi-Device Conversation:** Support multiple participants contributing to the same live conversation.
- **Audio Coordination:** Combine information from different devices to improve the quality and reliability of captured speech.
- **Speaker Identification:** Determine which participant is speaking, including during overlapping speech.
- **Noise & Overlap Handling:** Maintain usable transcription in realistic environments with background noise and simultaneous speakers.
- **Low-Latency Captions:** Produce readable live captions while handling corrections as more audio becomes available.
- **Session Continuity:** Handle participants joining, leaving, reconnecting, or temporarily losing connectivity.
- **Evaluation:** Demonstrate performance across different numbers of speakers and noise conditions using transcription accuracy, speaker attribution, and latency.

3. Fair Drop: Selling 500 Seats to 50,000 People Without Letting Bots Win

Backstory:

High-demand ticket sales, event registrations, and limited-seat drops often reward whoever has the fastest connection or most aggressive automation. Bots can send thousands of requests and overwhelm systems during launch, while legitimate users face timeouts, queues, or inconsistent allocation. A system designed around speed can become a competition between automated clients rather than people.

Problem Statement:

Develop Fair Drop, a high-demand sale and registration platform where automated clients cannot gain a significant advantage through speed, request volume, or repeated attempts. The system must remain reliable during a simulated flash crowd while maintaining consistent allocation and user state. Teams must demonstrate the behaviour of their system under both normal and adversarial traffic and provide measurable evidence of how allocation changes under different attack conditions.

Key Features:

- **High-Concurrency Support:** Handle large numbers of simultaneous users competing for limited availability.
- **Abuse Handling:** Detect and respond to automated clients, request flooding, and repeated attempts.
- **Allocation Integrity:** Prevent duplicate allocations, overselling, and inconsistent inventory state.
- **Reliable Sessions:** Maintain user state across refreshes, reconnects, and temporary failures.
- **Adversarial Testing:** Evaluate the system against configurable automated clients and varying traffic patterns.
- **Fairness Measurement:** Quantify allocation outcomes, system performance, and the effect of adversarial behaviour.



Google Developer Groups

On Campus • Fr. Conceicao Rodrigues College of Engineering

AROUND THE WORLD

BIT N BUILD



AI / ML

1. TrustLayer: AI-Powered Digital Authenticity and Trust

Backstory:

Generative AI has made digital content increasingly difficult to authenticate. Images, voices, videos, messages, and documents can be individually convincing while inconsistencies between them reveal a coordinated manipulation. Existing detection approaches often analyze each piece independently and may fail when faced with unfamiliar manipulation techniques.

Problem Statement:

Develop an AI system that analyzes multiple forms of digital content and determines whether they represent authentic, manipulated, or coordinated synthetic content.

The system should reason across multiple inputs rather than treating each artifact independently. It must identify the evidence behind its decisions, handle incomplete or conflicting information, and demonstrate how well it generalizes to manipulation patterns that were not directly represented during training.

Key Features:

- **Multimodal Analysis:** Analyze multiple forms of digital content within a single investigation.
- **Cross-Modal Reasoning:** Identify relationships and inconsistencies between different inputs.
- **Manipulation Detection:** Detect synthetic or altered content across the supported modalities.
- **Explainable Decisions:** Provide evidence supporting the system's classification.
- **Generalization:** Evaluate performance on manipulation techniques or combinations not seen during training.
- **Uncertainty Handling:** Distinguish between confident decisions and cases where the available evidence is insufficient.
- **Evaluation:** Measure detection performance across authentic, manipulated, and coordinated multimodal examples.

2. Black Box: A Flight Recorder for AI Agents

Backstory:

AI agents solve tasks through long execution graphs involving model calls, tool calls, retrieved context, and changing state. A run may contain many successful steps but fail because of one incorrect intermediate decision. Traditional traces can show what happened, but identifying the responsible step across thousands of executions is difficult.

Problem Statement:

Develop Black Box, an AI-powered debugging system that learns from agent execution traces to identify suspicious or failure-causing steps in an agent's execution.

The system should capture observable execution history and train a model to recognize patterns associated with successful and failed runs. When a new execution fails, the system should identify the most likely problematic step and allow that point to be investigated through replay and alternative execution paths.

The system must demonstrate that its learned diagnosis can be evaluated against known failures and that proposed fixes can be tested without unnecessarily repeating unaffected parts of the execution.

Key Features:

- **Execution Data:** Capture relevant information from successful and failed agent runs.
- **Failure Diagnosis:** Train a model to identify anomalous or failure-causing execution steps.
- **Failure Explanation:** Provide evidence from the execution history supporting the diagnosis.
- **Checkpointed Replay:** Allow executions to be investigated from intermediate states.
- **Alternative Execution:** Test changes to a suspected step and observe their effect on the final outcome.
- **Model Evaluation:** Measure the model's ability to localize known failures and generalize to previously unseen failures.
- **Trace Comparison:** Compare executions to understand how changes affected the final outcome.

3. Re:Learn: Adaptive Multimodal Learning Environment

Backstory:

Most Learning Management Systems follow a fixed sequence of lectures, PDFs, videos, assignments, and quizzes. Even adaptive systems often change only question difficulty rather than how a concept is taught.

A wrong answer is usually treated as simply incorrect, even though it may reveal a specific misconception. A student may know the equations but misunderstand the concept, or apply a programming rule incorrectly. Showing the correct solution does not necessarily fix the underlying misunderstanding.

Problem Statement:

Develop Re:Learn, an AI-powered learning system that trains a model to analyze a learner's answers, working, or code and identify the underlying misconception rather than simply marking the response as incorrect.

Teams must choose one domain: introductory programming, algebra, or physics. They must build and evaluate a model capable of distinguishing between different misconceptions that can produce similar incorrect answers. The system should use the model's diagnosis to generate an intervention targeted to the identified misconception.

After the intervention, the system must reassess the learner and determine whether the diagnosed misconception has actually been resolved, rather than assuming that a correct follow-up answer means learning has occurred.

Key Features:

- **Misconception Dataset:** Build a dataset of correct responses, incorrect responses, and their underlying misconceptions.
- **Misconception Model:** Train and evaluate a model to classify or infer the misconception behind a learner's response.
- **Misconception Differentiation:** Distinguish between different misconceptions that produce similar mistakes.
- **Adaptive Intervention:** Generate or select an intervention based on the model's diagnosis.
- **Resolution Assessment:** Determine whether the learner's underlying misconception has been resolved after intervention.
- **Learner Model:** Track recurring misconceptions and demonstrated understanding across attempts.
- **Model Evaluation:** Evaluate diagnosis accuracy and performance on responses or misconceptions not seen during training.



Google Developer Groups
On Campus • Fr. Conceicao Rodrigues College of Engineering

AROUND THE WORLD



Blockchain

1. Quorum: Don't Trust the Binary, Trust the Builders

Backstory:

Modern software depends on thousands of open-source packages, yet users generally trust that a published binary was actually built from the public source code. A compromised maintainer account, build server, or release pipeline can silently introduce malicious changes into widely distributed software. The 2024 XZ Utils incident demonstrated how a sophisticated supply-chain attack can remain hidden inside a trusted software ecosystem.

The core problem is that users have little practical way to independently verify that a released artifact actually corresponds to its claimed source.

Problem Statement:

Develop Quorum, a decentralized software supply-chain verification system where independent builders reproduce software releases from a pinned source commit and publish signed attestations of the resulting artifact. Consumers should be able to verify these attestations against configurable quorum policies before installing a release. The system must detect disagreements between builders and provide an auditable record of the verification process. A working end-to-end demonstration using a real open-source package is expected.

Key Features:

- **Independent Verification:** Allow software releases to be verified by multiple independent sources.
- **Artifact Integrity:** Establish whether a released artifact corresponds to its claimed source.
- **Disagreement Handling:** Detect and surface conflicting verification results.
- **Trust Assessment:** Allow consumers to determine whether available verification evidence is sufficient to trust a release.
- **Tamper Resistance:** Maintain a verifiable history of release verification and changes.
- **Release Decision:** Provide a clear mechanism for accepting or rejecting releases based on available evidence.
- **Evaluation:** Demonstrate the system using both valid releases and intentionally compromised or inconsistent artifacts.

2. Heirloom: Trust-Minimized Digital Inheritance

Backstory:

People increasingly hold important assets digitally, including encrypted documents, credentials, cryptocurrency wallets, personal records, and other sensitive information. When an owner dies or becomes permanently unavailable, legitimate heirs may have no reliable way to recover these assets. At the same time, releasing them based on a simple inactivity timer creates a serious risk of premature disclosure.

Problem Statement:

Develop Heirloom, a trust-minimized digital inheritance protocol that securely transfers protected digital assets to designated beneficiaries when predefined incapacity or unavailability conditions are satisfied. The system must prevent both unauthorized early access and permanent loss while ensuring that no single platform, guardian, or administrator can unilaterally access the owner's secrets.

Key Features:

- **Secure Access Control:** Ensure protected assets cannot be accessed before the required conditions are satisfied.
- **Unavailability Detection:** Determine when the conditions for initiating recovery have been met without relying on a single signal.
- **Recovery Authorization:** Require sufficient independent evidence before assets can be released.
- **Emergency Intervention:** Allow legitimate cancellation or intervention during an active recovery process.
- **Conditional Access:** Support different access conditions for different protected assets or beneficiaries.
- **Failure Handling:** Continue functioning when an authorization party becomes unavailable or unresponsive.
- **Auditability:** Maintain a verifiable history of recovery attempts, approvals, cancellations, and releases.

3. ModelLedger: Prove Which AI Model Produced This

Backstory:

AI-generated content increasingly passes through multiple models and applications. An image may be generated by one model, edited or upscaled by another, and published through a different platform. While digital signatures can prove who made a provenance claim, they do not by themselves prove that the claimed model actually produced the artifact. Transformations such as re-encoding, resizing, or editing can also break direct file bindings.

Problem Statement:

Develop ModelLedger, a blockchain-based system for establishing and verifying the provenance of AI-generated content.

The system should allow a verifier to determine whether an artifact's claimed origin and modification history can be trusted, even when the content has passed through multiple models or applications. It should distinguish between trusted, self-asserted, and unverifiable provenance while allowing verification without exposing private prompts or source files.

Key Features:

- **Provenance Verification:** Establish and verify the claimed origin and history of an artifact.
- **Provenance Trust:** Determine whether a provenance claim is supported by verifiable evidence or should be treated as untrusted.
- **Transformation Handling:** Preserve provenance when content undergoes legitimate modifications or format changes.
- **Multi-System Provenance:** Support content that passes through multiple AI systems or applications.
- **Tamper Detection:** Identify inconsistencies between an artifact and its claimed history.
- **Privacy-Preserving Verification:** Verify provenance without exposing sensitive generation data.
- **Adversarial Testing:** Demonstrate how the system handles fabricated, incomplete, or conflicting provenance claims.